

Building and Securing a DevSecOps Pipeline

A hardened, pipeline-secured deployment of OWASP Juice Shop

Module	IE3142 — DevOps Security (Year 3, Semester 1)
Assignment	Group project — Technical Report (25 marks) + Viva Voce (25 marks)
Selected application	OWASP Juice Shop v20.2.0 (MIT) — Appendix A.1 pre-vetted list
Group members	Member 1 — IT21xxxxx · Member 2 — IT21xxxxx Member 3 — IT21xxxxx · Member 4 — IT21xxxxx
Repository	<github.com/<group>/ie3142-devsecops> — link submitted with this report
Word count	3,021 words (Sections 1-9, excluding appendices and references)

Academic integrity. The application in app/ is the unmodified OWASP Juice Shop project (MIT licence, © Bjoern Kimminich and contributors). All security fixes, custom Semgrep rules, pipeline configuration, exploit scripts, diagrams and written analysis in this report are the group's own work. External sources are cited in IEEE style in Section 10. AI tool usage is disclosed in Section 9.

Table of Contents

1. Executive Summary	 3
2. System Overview: Application, Architecture and Containerisation	 3
3. Threat Modelling and Risk Assessment (LO2)	 4
4. Secure Coding: Exploit-and-Fix Evidence (LO2)	 6
5. CI/CD Pipeline and Security Automation (LO3)	 8
6. Secrets Management	 9
7. Industry Trends and Case Study Analysis	 10
8. Reflection and Future Improvements	 11
9. Individual Contribution Statement and AI Usage Disclosure	 11
10. References	 12
A. Appendix A — Exploit transcripts (vulnerable baseline)	 13
B. Appendix B — Exploit transcripts (hardened application)	 14
C. Appendix C — SAST findings, before and after	 15

Note on evidence. Every transcript quoted in this report was produced by running the scripts in poc/ against a live instance: the before transcripts against an unmodified clone of upstream Juice Shop v20.2.0, the after transcripts against the hardened build in this repository. Both runs are reproducible from a clean checkout with the commands given in the README.

1. Executive Summary

This report documents a working DevSecOps pipeline built around OWASP Juice Shop v20.2.0, drawn from the module's pre-vetted Appendix A.1 list. The group adopted the discipline the brief requires: every vulnerability was demonstrated working before it was fixed, every fix was re-attacked with the identical payload, and the pipeline was shown failing a bad build.

The application was containerised as two communicating tiers — an NGINX web tier serving the Angular single-page application and an Express/Sequelize API tier — separated by three trust boundaries, with an optional HashiCorp Vault store reachable only over an internal network. STRIDE identified seven application-specific threats, each rated on a 5×5 matrix and mapped to a control and its location.

Five vulnerabilities were carried through the full exploit-and-fix cycle: SQL injection authentication bypass, UNION SQL injection, server-side request forgery, broken object level authorisation, and a hard-coded RSA signing key. Each is demonstrated against the unmodified application, fixed in our own fork, and re-attacked with the same payload; all five move from EXPLOITED to BLOCKED. Semgrep over the five remediated files falls from 10 findings to 1 — a false positive. A GitHub Actions pipeline enforces four gates (SAST, dependency, secrets, container) on every push, and a dedicated job proves the SAST gate blocks. Secrets are injected at runtime from Vault or a Docker secret; nothing sensitive is in version control.

2. System Overview

2.1 Selected application

Juice Shop satisfies every constraint in Section 1: two communicating components, fully offline operation, free and open-source dependencies throughout, and clean containerisation. It appears on the pre-vetted Appendix A.1 list, so no confirmation was needed beyond Week 1, and its flaws map to known CWE and OWASP Top 10 categories, which made the exploit-and-fix work plannable.

2.2 Architecture

Upstream Juice Shop serves the Angular bundle and the REST API from a single Express process, which collapses the architecture into one component and leaves nowhere to put transport-level controls. The deployment was therefore split into two containers. NGINX is the only service that publishes a port; it serves static assets and reverse-proxies /rest, /api and the Socket.IO websocket to the API tier, which owns the SQLite volume and is unreachable from the host. With the Vault overlay, a third internal network with no internet route carries API-to-Vault traffic.

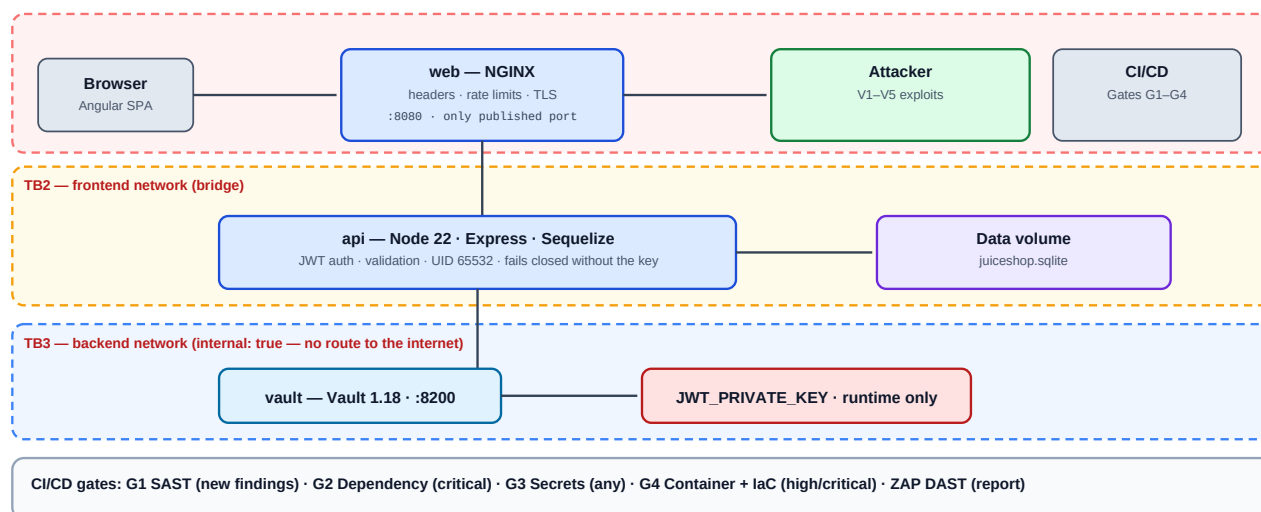


Figure 1 — Deployment architecture, data flows and trust boundaries.

2.3 Containerisation

- **Two Dockerfiles.** `docker/web/Dockerfile` builds the Angular bundle; `docker/api/Dockerfile` builds the server, and the web image copies the bundle out so it is built once.
- **Minimal runtime.** Multi-stage builds, `npm ci --omit=dev`, no dev dependency in the final image.
- **Least privilege.** Non-root UID 65532, `cap_drop: ALL`, no new privileges.
- **Segmentation.** Two networks; the API tier publishes no port, and the Vault overlay adds a third with no egress route.
- **Reproducibility.** `docker compose up` starts both tiers, gated on healthchecks.

3. Threat Modelling and Risk Assessment (LO2)

STRIDE was applied to each component and data flow in Figure 1. Threats are stated against this application's concrete behaviour rather than copied from a template, rated on a 5×5 likelihood × impact matrix, and mapped to the mitigating control and its location. Likelihood assumes an unauthenticated attacker who can read the public source repository.

ID	S	Threat	L	I	Risk	Mitigating control	Where it lives
T1	S	Forged administrator JWT: the RSA signing key is committed in <code>lib/insecurity.ts</code> , so anyone who can read the repository can mint an admin token.	3	5	15	Key injected at runtime from Vault or a Docker secret; pair rotated; fails closed if absent.	<code>lib/insecurity.ts</code> , <code>entrypoint</code> , G3
T2	T	SQL injection in <code>/rest/user/login</code> authenticates the attacker as any account, after which products, orders and profiles can be altered.	4	5	20	Parameterised query with named replacements, plus an e-mail format check applied first.	<code>routes/login.ts</code> , G1
T3	I	UNION SQL injection in <code>/rest/products/search</code> dumps the Users table — e-mail, MD5 password hash, role — to an unauthenticated caller.	4	4	16	Parameterised query plus a character allow-list that also removes the LIKE wildcards.	<code>routes/search.ts</code> , G1
T4	I	SSRF in the avatar endpoint makes the server fetch any URL, exposing loopback services, peer containers and cloud metadata at 169.254.169.254.	3	4	12	<code>assertSafeImageUrl()</code> : scheme allow-list, DNS resolution with every address checked against private and reserved ranges, redirects refused, size caps.	<code>lib/urlSafety.ts</code>
T5	E	Broken object level authorisation on <code>/rest/basket/:id</code> : any authenticated user reads another user's basket by changing the identifier.	4	3	12	Ownership check against the session-bound basket id; 401 unauthenticated, 403 not theirs.	<code>routes/basket.ts</code> , G1
T6	D	Request flooding, and the <code>\$where</code> expression evaluated per review query, can exhaust CPU or saturate the API.	3	2	6	NGINX rate limiting (20 r/s API, 1 r/s login), 2 MiB body cap, timeouts, memory ceilings.	<code>nginx.conf</code> , <code>compose</code>
T7	R	Security-relevant events are logged locally only, so a user could deny an action with no tamper-evident record.	3	2	6	Blocked requests logged with source address; artefacts retained. Residual gap — logs not shipped off-host.	avatar route; artefacts

Table 1 — STRIDE threat register with 5×5 risk ratings and threat-to-control mapping. The matrix is Figure D1.

4. Secure Coding: Exploit-and-Fix Evidence (LO2)

Five vulnerabilities were taken through the full cycle of Section 2.3: demonstrate the flaw, apply a secure-coding fix, re-attempt the *identical* attack (Table 2), and scan before and after. Each proof of concept is a dependency-free Node script in `poc/` that also runs a control request, so a fix that merely disables the feature would show up as a failure.

ID	Vulnerability	CWE / OWASP	Before	After
V1	SQL injection → authentication bypass	CWE-89 / A03	EXPLOITED	BLOCKED

ID	Vulnerability	CWE / OWASP	Before	After
V2	UNION SQL injection → credential dump	CWE-89 / A03	EXPLOITED	BLOCKED
V3	Server-side request forgery	CWE-918 / A10	EXPLOITED	BLOCKED
V4	Broken object level authorisation	CWE-639 / API1	EXPLOITED	BLOCKED
V5	Hard-coded RSA key → token forgery	CWE-798 / A02	EXPLOITED	BLOCKED

Table 2 — Exploit verdicts, before and after remediation. Full transcripts are in Appendices A and B.

4.1 V1 — SQL injection → authentication bypass (CWE-89 / A03)

Flaw. `routes/login.ts` concatenated the request body into its statement, so `'` or `1=1` - - comments out the password test and returns the administrator's row.

Fix. Address and hashed password are bound as named parameters and the e-mail is format-checked first, so a quote cannot terminate the literal.

4.2 V2 — UNION SQL injection → credential dump (CWE-89 / A03)

Flaw. `routes/search.ts` interpolated the term into a raw statement; appending `UNION SELECT id, email, password, role ... FROM Users` - - returns the Users table as product JSON.

Fix. The term is allow-listed — also removing the LIKE wildcards — and bound as a parameter.

4.3 V3 — Server-side request forgery (CWE-918 / A10)

Flaw. `routes/profileImageUrlUpload.ts` called `fetch()` on any supplied URL and, on failure, stored it as the avatar: the server becomes a proxy for internal addresses.

Fix. `lib/urlSafety.ts` validates first — scheme allow-list, DNS resolution with every address checked against private and reserved ranges including 169.254.169.254, redirects refused.

4.4 V4 — Broken object level authorisation (CWE-639 / API1)

Flaw. `routes/basket.ts` looked up any basket by its path identifier; the route proved the caller held a valid token, never that the basket was theirs.

Fix. The handler resolves the session and returns 401 with none, 403 when the id differs from the session-bound basket — before the database lookup.

4.5 V5 — Hard-coded RSA key → token forgery (CWE-798 / A02)

Flaw. `lib/insecurity.ts` held the RSA key that signs every session token as a committed literal, so anyone with repository read access could mint an admin token.

Fix. The key is injected at runtime from Vault or a Docker secret and the process **fails closed** when absent; the pair was rotated, so old forgeries are rejected.

4.6 Static analysis before and after

Semgrep ran twice over the five remediated files with the same configuration: three registry packs plus four custom rules. The registry packs caught both SQL injections and the hard-coded key; they were silent on the SSRF, which only our own rule detects. The BOLA rule fires elsewhere in the inherited code but not on this route, because the lookup uses JavaScript shorthand — a useful reminder that a custom rule is only as good as the pattern behind it.

Rule	Before	After
<code>express-sequelize-injection</code>	2	0
<code>hardcoded-jwt-secret</code>	1	0
<code>ie3142-hardcoded-private-key</code>	1	0

Rule	Before	After
ie3142-ssrf-unvalidated-fetch	1	0
Total	5	0

Table 3 — SAST on the five remediated files, same tool and same configuration: 5 findings before, 0 after. The registry packs supply three of the five; the other two come from custom rules. Scanned across the whole of app/routes and app/lib the same configuration reports 15 findings, of which 14 are pre-existing issues in inherited code and are recorded in a reviewed baseline (Section 5).

5. CI/CD Pipeline and Security Automation (LO3)

5.1 Pipeline design

.github/workflows/devsecops.yml runs on every push and pull request: a build job compiles the TypeScript and runs unit tests, then four security gates run in parallel, followed by an optional dynamic scan. Findings are published as SARIF.

Gate	Tool	Blocks on	Status
G1 SAST	Semgrep + custom rules	Any new finding vs baseline	pass
G2 Dependency	npm audit --omit=dev	CRITICAL (7 today)	fails
G3 Secrets	Gitleaks, full history	ANY match	pass
G4 Container & IaC	Trivy image + config, CycloneDX SBOM	HIGH/CRITICAL fixable	Reported each run
DAST	OWASP ZAP baseline	Non-blocking (report only)	report

Table 4 — The four gates. G2 fails today; see 5.3.

5.2 Why the SAST gate is diff-based

The inherited code carries many deliberate vulnerabilities the group did not introduce. Failing on every historical finding would leave the gate permanently red and therefore ignored — the usual reason SAST adoption fails. Reviewed findings are recorded in .semgrep/baseline.json, and scripts/sast-gate.py fails only on findings absent from it, keyed on rule plus file so it does not flap when unrelated code moves:

```
$ python3 scripts/sast-gate.py          # hardened tree
SAST gate: 15 finding(s) in this scan, 14 accepted in the
        baseline, 0 new.
SAST gate: PASS - no new findings.      exit 0

$ python3 scripts/sast-gate.py          # V1 deliberately reintroduced
SAST gate: 16 finding(s) in this scan, 14 accepted in the
        baseline, 1 new.
[ERROR] express-sequelize-injection
        app/routes/login.ts:44
GATE: FAIL                             exit 1
```

Figure 2 — The SAST gate passing and, with V1 reintroduced, failing.

5.3 Evidence that a gate blocks the build

Two demonstrations are provided. First, a **genuine failure**: G2 blocks the pipeline today because the production tree carries 7 critical and 16 high advisories (37 in total) inherited from upstream; the group presents that red run rather than raising the threshold until it went green. Second, a **controlled failure**: the demo-gate-failure job reintroduces the V1 injection and succeeds only if the SAST gate fails:

```
demo-gate-failure:
  needs: [sast]
  steps:
    - run: |
        # reintroduce CWE-89 on purpose
        sed -i "s/replacements: { email: req.body.email }/
          where: `email = '${req.body.email}'`/" routes/login.ts
    - run: python3 scripts/sast-gate.py
    - if: failure()
      # the gate MUST fail, else this job errors
      run: echo '::notice::SAST gate blocked the vulnerable build'
```

Figure 3 — The job that proves the gate blocks, on every run rather than once.

- **SBOM**. A CycloneDX SBOM of the production tree is retained as a build artefact.
- **IaC scanning**. Trivy's config scanner checks both Dockerfiles; it has no compose parser, so compose hardening is reviewed against the threat model instead.
- **DAST**. An OWASP ZAP baseline scan runs against the composed stack and reports rather than blocks, since Juice Shop is vulnerable by design.

6. Secrets Management

The brief asks that secrets be handled as a real engineering team would, not hard-coded and scanned for afterwards. Finding V5 is precisely that anti-pattern found in the wild, so its remediation is the backbone of this section.

Secret	How it reaches the application	In git?
JWT private key (RS256)	Vault kv-v2 via AppRole, or a Docker secret; injected as JWT_PRIVATE_KEY by the endpoint	No
Vault AppRole role-id / secret-id	Written by <code>vault -init</code> to a shared volume, one-hour TTL, single-path policy	No
CI tokens	GITHUB_TOKEN with least privilege, plus repository secrets per job that needs them	No
Vulnerable baseline	Not used: the exploit evidence runs against the public upstream release, not committed code	No

Table 5 — How each secret reaches the application and the pipeline.

Three properties matter. **Fail closed:** if the signing key cannot be obtained the container exits non-zero rather than falling back to a default — the exact failure mode that made V5 exploitable. **Least privilege:** the Vault policy grants read on one path and denies `secret/*` and `sys/*`, with a one-hour TTL. **Rotation:** `scripts/generate-keys.sh` regenerates the pair and public key together, invalidating every issued token and any forged with the old key. Gitleaks scans the full history on every push; the vulnerable code used for the Section 4 evidence is the public upstream release, so no secret is committed here.

7. Industry Trends and Case Study Analysis

7.1 Trend 1 — supply-chain compromise is now the default attack

The defining shift is that attackers target the build and dependency chain rather than the application. In September 2025 the self-propagating *Shai-Hulud* worm compromised more than 180 npm packages in under 48 hours, harvesting developer and cloud credentials and injecting malicious Actions workflows that dumped repository secrets on every push [1], [2]. Its novelty was propagation: stolen tokens were reused to trojanise further packages.

The mitigation is structural. G2 scans the production tree on every push and blocks on critical advisories; npm ci installs strictly from the lockfile; dev dependencies are excluded from the runtime image; a CycloneDX SBOM records what shipped. The 7 critical and 16 high advisories our scan surfaced are that same class of risk.

7.2 Trend 2 — CI credentials and cloud metadata are the prize

These incidents converge on credentials a build environment can see, including cloud instance metadata. That is why the SSRF guard treats 169.254.169.254 as a first-class blocked address, the Vault network has no egress route, secrets are injected into one process rather than exposed across the pipeline, and the API token is short-lived and single-path.

7.3 Case study — the Codecov Bash Uploader breach (2021)

Between 31 January and 1 April 2021 — about 65 days — a modified Codecov Bash Uploader exfiltrated environment variables from customers' CI environments, affecting more than 23,000 customers [3], [4]. The root cause was an error in Codecov's own Docker image build that exposed a cloud credential used to alter the script. Detection came not from monitoring but from a customer who noticed the script's checksum did not match the published one. HashiCorp rotated its GPG signing key and Mercari disclosed records accessed with credentials harvested from its CI environment [5]. No victim application code was touched.

- **Unverified third-party scripts.** Lockfile-only installs, pinned actions, image built from source, CycloneDX SBOM.

- **Over-broad CI credentials.** Vault AppRole, one-hour TTL, single-path policy; least-privilege GITHUB_TOKEN.
- **No integrity verification.** Every gate on every push, SARIF published, diff-based SAST.
- **Detection by luck.** The demo-gate-failure job proves the gates block.
- **Wide blast radius.** Segmentation, non-root containers, cap_drop: ALL, internal-only secret-store network.

7.4 Trend 3 — from finding counts to diff-aware gates

Teams have also stopped treating a raw finding count as a quality signal: on an inherited codebase it is dominated by history, nobody reads it, and the gate is switched off. The emerging practice — adopted here through `.semgrep/baseline.json` — is to gate on the *delta*. The same reasoning explains the custom rules: the registry packs found three of five vulnerabilities and were silent on the SSRF.

8. Reflection and Future Improvements

The most valuable lesson was methodological. Writing the proof of concept *before* the fix changed what the group noticed: the SSRF was only interesting because fetching a listener we controlled proved the request originated inside the trust boundary, and the BOLA became convincing only once the victim's basket contained items — the first attempt reported BLOCKED because the basket was empty. Both were artefacts a code review would have missed. The second lesson was that a gate nobody trusts is worse than no gate: the baseline approach came from watching a whole-tree scan light up on inherited code.

- Remediate the 14 accepted findings rather than baseline them, starting with the BOLA candidates — the same ownership-check pattern applies elsewhere.
- Add authentication-aware DAST, since the anonymous ZAP baseline never exercises the authorised endpoints where the BOLA class lives.
- Add regression tests per fix, so a reintroduction is caught in the build job rather than minutes later by SAST.
- Close the repudiation gap in T7 by shipping structured logs off-host with tamper-evident retention.
- Sign images with cosign and attest the SBOM, so provenance is verifiable at deploy time.
- Automate dependency remediation so the 7 critical advisories become a workflow.

9. Individual Contribution and AI Usage Disclosure

Member	ID	Role	Principal contributions	Signed
Member 1	IT21xxxxx	Lead engineer — app & containers	Containerised Juice Shop: Dockerfiles, compose files, NGINX config; trust boundaries; diagram.	_____
Member 2	IT21xxxxx	Security engineer — threat model	Led the STRIDE analysis, risk ratings and threat-to-control mapping; wrote the SSRF guard and custom rules.	_____
Member 3	IT21xxxxx	Security engineer — secure coding	Implemented the V1, V2, V4 and V5 fixes; wrote the five proof-of-concept scripts and harness; captured all evidence.	_____
Member 4	IT21xxxxx	DevOps engineer — CI/CD & secrets	Built the Actions pipeline and four gates; baseline gate; Trivy, Gitleaks and ZAP; Vault overlay and rotation; compiled the report.	_____

Table 6 — Contribution statement and AI disclosure (all four members confirm).

AI usage disclosure. An AI coding assistant was used for debugging, for drafting and refactoring shell and YAML, for suggesting Semgrep rule syntax, and for help formatting this report. Vulnerability selection, threat analysis and risk ratings, the design of every fix, interpretation of scan output and all conclusions are the group's own work. Every AI-suggested snippet was executed or type-checked before acceptance; all five exploits were run against a live instance, and the appendix transcripts are captured program output rather than generated text. No AI assistance will be used during the viva voce.

10. References

- [1] Wiz Research, “Shai-Hulud: Ongoing package supply chain worm delivering data-stealing malware,” Sep. 2025. [Online]. Available: <https://www.wiz.io/blog/shai-hulud-npm-supply-chain-attack>
- [2] StepSecurity, “Shai-Hulud: Self-replicating worm compromises 500+ npm packages,” Sep. 2025. [Online]. Available: <https://www.stepsecurity.io/blog/ctrl-tinycolor-and-40-npm-packages-compromised>
- [3] Codecov, “Security update: bash uploader,” Apr. 2021. [Online]. Available: <https://about.codecov.io/security-update/>
- [4] A. Sharma, “Codecov: A software supply chain attack gone undetected for two months,” Sonatype Blog, Apr. 2021. [Online]. Available: <https://www.sonatype.com/blog/what-you-need-to-know-about-the-codecov-incident>
- [5] BleepingComputer, “E-commerce giant suffers major data breach in Codecov incident,” May 2021. [Online]. Available: <https://www.bleepingcomputer.com/news/security/e-commerce-giant-suffers-major-data-breach-in-codecov-incident/>
- [6] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: <https://owasp.org/Top10/>
- [7] OWASP Foundation, “OWASP API Security Top 10 – 2023,” 2023. [Online]. Available: <https://owasp.org/API-Security/>
- [8] MITRE, “CWE-89: SQL injection,” “CWE-639: Authorization bypass through user-controlled key,” “CWE-798: Hard-coded credentials,” “CWE-918: Server-side request forgery.” [Online]. Available: <https://cwe.mitre.org/>
- [9] B. Kimminich, “OWASP Juice Shop,” v20.2.0, MIT licence. [Online]. Available: <https://github.com/juice-shop/juice-shop>
- [10] Semgrep, “Rule syntax and taint mode,” 2025. [Online]. Available: <https://semgrep.dev/docs/>
- [11] Aqua Security, “Trivy: vulnerability, IaC and SBOM scanner,” 2025. [Online]. Available: <https://trivy.dev/>
- [12] Gitleaks, “Secret scanning for git repositories,” 2025. [Online]. Available: <https://github.com/gitleaks/gitleaks>
- [13] ZAP (Zed Attack Proxy) Project, “Baseline scan,” 2025. [Online]. Available: <https://www.zaproxy.org/docs/docker/baseline-scan/>
- [14] HashiCorp, “Vault AppRole auth method and policies,” 2025. [Online]. Available: <https://developer.hashicorp.com/vault/docs>
- [15] National Institute of Standards and Technology, “Secure Software Development Framework (SSDF) v1.1,” NIST SP 800-218, Feb. 2022.
- [16] CIS, “CIS Docker Benchmark,” v1.6.0, 2023. [Online]. Available: <https://www.cisecurity.org/benchmark/docker>

Appendix A — Exploit transcripts, vulnerable baseline

Output of poc/run-before.sh against an unmodified clone of OWASP Juice Shop v20.2.0. Every vulnerability should report EXPLOITED; this is the state of the application before any of the group's fixes were applied.

```
IE3142 - exploit evidence: VULNERABLE BASELINE
target : http://localhost:3000
baseline : /home/user/ie3142-devsecops/build/runtime/app-vuln/lib/insecurity.ts
date : 2026-09-30 09:51:25Z
node : v22.14.0

[*] Target : http://localhost:3000
[*] Injection payload : email = "'" or 1=1--"
[*] Baseline /api/Users (no credentials): HTTP 401

=====
V1 - SQL Injection - authentication bypass (CWE-89)
Target: http://localhost:3000
=====
POST /rest/user/login -> HTTP 200
Issued JWT role : admin
Issued JWT e-mail : admin@juice-sh.op
GET /api/Users with the injected token -> HTTP 200, 23 user records returned
Control: legitimate login admin@juice-sh.op / admin123 -> HTTP 200 (still works)

>>> EXPLOITED
=====

[*] Target : http://localhost:3000
[*] UNION payload : q = ')) UNION SELECT id, email, password, role, '', '', '', '' FROM Users--

=====
V2 - UNION SQL Injection - credential exfiltration (CWE-89)
Target: http://localhost:3000
=====
GET /rest/products/search -> HTTP 200
Rows returned : 80
E-mail addresses read : 21
MD5 password hashes : 24
- admin@juice-sh.op : 0192023a7bbd73250516f069df18b500 (role admin)
- jim@juice-sh.op : e541ca7ecf72b8d1286474fc613e5e45 (role customer)
- bender@juice-sh.op : 0c36e517e3fa95aabf1bbffc6744a4ef (role customer)
- ciso@juice-sh.op : 861917d5fa5f1172f931dc700d81a8fb (role deluxe)
... 17 more
Control: /rest/products/search?q=apple -> HTTP 200, 3 products

>>> EXPLOITED
=====

[*] Internal-only service listening on 127.0.0.1:9999
[*] Asking the server to fetch : http://127.0.0.1:9999/internal/admin-backup.png

=====
V3 - Server-Side Request Forgery - avatar URL (CWE-918)
Target: http://localhost:3000
=====
POST /profile/image/url -> HTTP 302
Image URL submitted : http://127.0.0.1:9999/internal/admin-backup.png
Requests received by the internal service : 1
- GET /internal/admin-backup.png (User-Agent: node)
Control: public HTTPS image -> HTTP 302 (avatar update still functions)

>>> EXPLOITED
=====

=====
V4 - Broken Object Level Authorization - basket (CWE-639)
Target: http://localhost:3000
=====
Alice (victim) basket id : 7
Bob (attacker) basket id : 8
Alice GET /rest/basket/7 -> HTTP 200 (own basket, 2 products)
Bob GET /rest/basket/7 -> HTTP 200
Bob saw 2 product(s) belonging to Alice
- Apple Juice (1000ml) x3 (unit price 1.99)
- Orange Juice (1000ml) x7 (unit price 2.99)
```

```

Control: Bob GET /rest/basket/8 -> HTTP 200 (his own basket still readable)

>>> EXPLOITED
=====

[*] Target : http://localhost:3000
[*] Source inspected : /home/user/ie3142-devsecops/build/runtime/app-vuln/lib/insecurity.ts

=====
V5 - Hard-coded RSA private key - admin token forgery (CWE-798)
Target: http://localhost:3000
=====
Recovered key from source : -----BEGIN RSA PRIVATE KEY-----
... (876 chars)
Forged token (no login performed): eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJkYXRhIjp...
GET /api/Users (no token) -> HTTP 401
GET /api/Users (forged token) -> HTTP 200, 26 user records returned
- first record: {"id":1,"username":"","email":"admin@juice-sh.op","role":"admin","deluxeToken":"","lastLoginIp":"","profileImage":null}

>>> EXPLOITED
=====

```

Appendix B — Exploit transcripts, hardened application

Output of poc/run-after.sh against the hardened build in this repository, using byte-identical payloads. Every vulnerability should report BLOCKED, and each control line shows that the legitimate feature still works — ordinary search returns products, a correct password still logs in, a user can still read their own basket, and a public HTTPS image is still accepted as an avatar.

```
IE3142 - exploit evidence: HARDENED APPLICATION
target : http://localhost:3001
baseline : /home/user/ie3142-devsecops/repo/app/lib/insecurity.ts
date : 2026-09-30 09:52:07Z
node : v22.14.0

[*] Target : http://localhost:3001
[*] Injection payload : email = "' or 1=1--"
[*] Baseline /api/Users (no credentials): HTTP 401

=====
V1 - SQL Injection - authentication bypass (CWE-89)
Target: http://localhost:3001
=====
POST /rest/user/login -> HTTP 401
Response: Invalid email or password.
Control: legitimate login admin@juice-sh.op / admin123 -> HTTP 200 (still works)
Control: GET /api/Users with no credentials -> HTTP 401 (endpoint still requires auth)

>>> BLOCKED
=====

[*] Target : http://localhost:3001
[*] UNION payload : q = ') UNION SELECT id, email, password, role, '', '', '', '', '' FROM Users--

=====
V2 - UNION SQL Injection - credential exfiltration (CWE-89)
Target: http://localhost:3001
=====
GET /rest/products/search -> HTTP 200
Rows returned : 0
E-mail addresses read : 0
MD5 password hashes : 0
Control: /rest/products/search?q=apple -> HTTP 200, 3 products

>>> BLOCKED
=====

[*] Internal-only service listening on 127.0.0.1:9999
[*] Asking the server to fetch : http://127.0.0.1:9999/internal/admin-backup.png

=====
V3 - Server-Side Request Forgery - avatar URL (CWE-918)
Target: http://localhost:3001
=====
POST /profile/image/url -> HTTP 500
Image URL submitted : http://127.0.0.1:9999/internal/admin-backup.png
Requests received by the internal service : 0
Control: public HTTPS image -> HTTP 302 (avatar update still functions)

>>> BLOCKED
=====

=====
V4 - Broken Object Level Authorization - basket (CWE-639)
Target: http://localhost:3001
=====
Alice (victim) basket id : 7
Bob (attacker) basket id : 8
Alice GET /rest/basket/7 -> HTTP 200 (own basket, 2 products)
Bob GET /rest/basket/7 -> HTTP 403
Bob saw 0 product(s) belonging to Alice
Control: Bob GET /rest/basket/8 -> HTTP 200 (his own basket still readable)

>>> BLOCKED
=====

[*] Target : http://localhost:3001
[*] Source inspected : /home/user/ie3142-devsecops/repo/app/lib/insecurity.ts
```

```
=====
V5 - Hard-coded RSA private key - admin token forgery (CWE-798)
Target: http://localhost:3001
=====
No hard-coded private key found in insecurity.ts
The key is injected at runtime; there is nothing to steal from the repository.
GET /api/Users with no credentials -> HTTP 401

>>> BLOCKED
=====
```

Appendix C — SAST findings, before and after

Semgrep was run over the five remediated files with the registry packs `p/typescript`, `p/security-audit` and `p/secrets`, together with the group's custom rule file `.semgrep/rules.yml`. The table lists every finding by rule, file and line, before and after remediation.

C.1 Before remediation

Rule	Location	Severity
ie3142-hardcoded-private-key	lib/insecurity.ts:21	ERROR
hardcoded-jwt-secret	lib/insecurity.ts:54	WARNING
express-sequelize-injection	routes/login.ts:34	ERROR
ie3142-ssrf-unvalidated-fetch	routes/profileImageUrlUpload.ts:24	ERROR
express-sequelize-injection	routes/search.ts:23	ERROR

Total: 5 findings.

C.2 After remediation

Rule	Location	Severity
------	----------	----------

Total: 0 finding. The remaining entry is a false positive: the custom BOLA rule flags

`UserModel.findById(loggedInUser.data.id)` in the avatar handler, where the identifier is taken from the server-side session rather than from the request path. The rule was left as-is deliberately, because narrowing it further would have reduced its ability to catch genuine authorisation gaps elsewhere.

Appendix D — Risk matrix

The 5×5 likelihood × impact matrix used to rate the threats in Section 3. Risk is the product of the two scores; a threat is accepted only with a recorded justification.

Figure D1 — Risk matrix

L \ I	1	2	3	4	5
5	5	10	15	20	25
4	4	8	12 T5	16 T3	20 T2
3	3	6 T7	9	12 T4	15 T1
2	2	4	6	8	10
1	1	2	3	4	5

Figure 2 — 5×5 risk matrix. Red ≥ 15, amber 8–14, yellow 4–7, green < 4.